Fraudulent Review Detection System

AI/ML Backend Assignment (NLP + Scalable Inference)

Overview

This assignment challenges you to build a scalable ML-powered review moderation service for an e-commerce or review platform (inspired by Amazon or Flipkart's spam detection). The system ingests user reviews, detects fraudulent ones using NLP-based anomaly detection and behavioral heuristics, and integrates with a backend API for real-time scoring and quarantine.

Real-world complexity: Handle 1M+ reviews/day with sub-200ms inference latency; combat model drift (e.g., evolving spam bots using AI-generated text); address data imbalance (95% legitimate reviews); and ensure explainability for human moderators (e.g., "High score due to repetitive phrasing + high post velocity").

Use a Node/Python backend for API orchestration, MongoDB for storage, and Hugging Face for NLP models. *(Optional: Add a simple React dashboard for monitoring).*

We prioritize **ML engineering:** Fine-tuning, deployment optimizations (ONNX quantization), drift monitoring, and seamless backend integration over perfect accuracy.

Exact Problem Statement & Expected Workflow

Your primary objective is to build the Review Guard moderation service. Here is exactly what you need to do:

1. **Dataset Generation & Preprocessing:** Run the provided Faker script to generate the imbalanced dataset of 50k reviews. Write a robust preprocessing pipeline to tokenize the text for BERT, and engineer numeric features (like `velocity`). You must handle the extreme class imbalance (95% legit) by implementing SMOTE or a visually demonstrated alternative technique on the training set.
2. **Model Fine-Tuning:** Take a pre-trained language model (e.g., `distilbert-base-uncased` from Hugging Face) and fine-tune it for binary classification (Fraud vs Legit). You must concatenate the NLP embeddings with the `velocity` heuristic value before the final classification layer.
3. **High-Speed Inference API:** Export the trained model to ONNX format and quantize it to INT8 to maximize CPU inference speed. Build a FastAPI (or Express) backend that exposes a `/api/reviews/detect/batch` endpoint. This endpoint must successfully process incoming batches of reviews in under 200ms on average.
4. **Drift Monitoring Engine:** Implement a backend cron job or Celery worker that continuously compares the distribution of live incoming prediction scores against the original training data distribution using a Kolmogorov-Smirnov (KS) test. It must trigger a mock alert if significant data drift is detected ($p < 0.05$).

Gokhale Business Bay, A-1111, Paschimnagari, Kothrud, Pune - 411038, Maharashtra, India.
www.embeds2.com

Embed Square Solutions Pvt. Ltd.

Complications

- **Imbalance & Drift:** 95% legit data → use SMOTE oversampling; monitor Kolmogorov-Smirnov (KS) test on live vs. training distributions (alert if $p<0.05$).
- **Inference Scale:** Batch up to 100 reviews/sec; optimize with ONNX Runtime for CPU (<200ms avg).
- **Integration:** Webhook for review submission; store in Mongo with audit logs; handle partial failures (e.g., model down → fallback to rules).
- **Ethics:** Bias mitigation (e.g., non-English reviews); explainable AI via LIME/SHAP for top features.

Your System Must:

5. Train/fine-tune a model on synthetic/realistic data.
6. Expose a fast API for inference.
7. Persist reviews and scores; monitor drift.
8. *(Optional)* Provide a dashboard for score visualization and alerts.

Dataset (Synthetic Generation Script)

Run this Python script once to generate `reviews_dataset.csv` (50k reviews, balanced fraud:legit ~1:19):

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from faker import Faker
import random

fake = Faker()
np.random.seed(42)

n_reviews = 50_000
fraud_rate = 0.05  # 5% fraud for imbalance

data = []
start = datetime(2024, 1, 1)

for i in range(n_reviews):
    user_id = random.randint(1, 10_000)
    post_date = start + timedelta(days=random.randint(0, 365))
    velocity = np.random.poisson(2)  # Avg 2 posts/day; fraud >5
    is_fraud = random.random() < fraud_rate
```

Embed Square Solutions Pvt. Ltd.

Gokhale Business Bay, A-1111, Paschimnagari, Kothrud, Pune - 411038, Maharashtra, India.
www.embeds2.com

```
if is_fraud:
        text = fake.sentence(nb_words=15, ext_word_list=["amazing", "best eve
r", "5 stars"]) * random.randint(2, 4)  # Repetitive spam
        rating = 5
        velocity = np.random.poisson(8)  # High velocity for fraud
    else:

 text = fake.paragraph(nb_sentences=3)
        rating = random.randint(1, 5)

    data.append({
        'review_id': i + 1,
        'user_id': user_id,
        'text': text,
        'rating': rating,
        'velocity': velocity,
        'timestamp': post_date.isoformat(),
        'is_fraud': is_fraud  # Ground truth for training
    })

df = pd.DataFrame(data)
df.to_csv('reviews_dataset.csv', index=False)
print(f"Generated {n_reviews} reviews (fraud: {df['is_fraud'].sum()})")
```

Columns:

Column	Description	Type
`review_id`	Unique ID	Integer
`user_id`	Submitter's ID	Integer
`text`	Review content	String
`rating`	Star rating (1–5)	Integer
`velocity`	Posts in last 24h	Integer
`timestamp`	Submission time	ISO Date
`is_fraud`	Label (train only)	Boolean

Split: 80% train, 10% val, 10% test. Use for fine-tuning.

Tech Stack

- **Backend:** Python (FastAPI preferred) or Node.js (Express); integrate Transformers via subprocess or TensorFlow.js.
- **ML:** Hugging Face Transformers (`distilbert-base-uncased`); scikit-learn for heuristics/SMOTE; ONNX for export.
- **DB:** MongoDB (Mongoose/PyMongo); collection for reviews with indexes on `user_id/timestamp`.

Gokhale Business Bay, A-1111, Paschimnagari, Kothrud, Pune - 411038, Maharashtra, India.
www.embeds2.com

Embed Square Solutions Pvt. Ltd.

- **Extras:** Redis (caching scores), Celery (drift cron).
- *(Optional)* **React:** For dashboard with Recharts.

Tasks / Functional Requirements

1. ML Pipeline

- **Preprocessing:** Tokenize text (max 512 tokens); engineer features (e.g., text length, sentiment polarity via VADER).

- **Model Training:**
  - Fine-tune DistilBERT on binary classification (fraud/legit); add velocity as numeric feature.
  - Handle imbalance: SMOTE on train set.
  - Eval: AUC-ROC >0.92 on val; F1 >0.85 for fraud class.
  - Export to ONNX; quantize for speed (int8).
- **Inference:** Batch endpoint; combine NLP score (0–1) + rules (e.g., `velocity_penalty = min(velocity/10, 1)`).
- **Drift Monitoring:** Cron job (every hour): KS-test on live scores vs. training dist; alert via email sim (print/log) if drift.

2. Backend API (FastAPI/Express)

Schema (Mongo / PyMongo Example):

```
db.reviews.insert_one({
    'review_id': str(uuid.uuid4()),
    'user_id': user_id,
    'text': text,
    'score': 0.75,  # Fraud prob
    'explanation': ['High velocity', 'Anomalous sentiment'],
    'status': 'quarantined',  # if score > 0.7
    'timestamp': datetime.utcnow()
})
```

Endpoints:

- `POST /api/reviews/submit` (body: `{text, user_id, rating}`); queue for async scoring.
- `POST /api/reviews/detect/batch` (array of reviews); return `[{id, score, explanation}]`.
- `GET /api/reviews/:id` (fetch with score).
- `GET /api/drift/status` (KS p-value, recent drift events).

Optimizations: Redis cache for user velocity (TTL 1d); Mongo transactions for score updates.

Gokhale Business Bay, A-1111, Paschimnagari, Kothrud, Pune - 411038, Maharashtra, India.
www.embeds2.com

Embed Square Solutions Pvt. Ltd.

3. Integration & Monitoring

- **Queue:** Celery/BullMQ for async scoring (process 1000s/sec).
- **Alerts:** If drift, log + notify (e.g., Slack webhook sim).
- **Testing:** Load test 10k reviews; measure latency/AUC.

4. Optional Dashboard (React)

- Table of recent reviews (TanStack Query for fetching); filter by score/user.
- Charts: Fraud rate over time (Recharts line).

Deliverables

- **GitHub repo:** `ReviewGuard_[YourName]`

- **README.md:**
  - Setup (`docker-compose` for Mongo/Redis; `pip install -r requirements.txt`).
  - Training/run instructions (e.g., `python train.py --data reviews_dataset.csv`).
  - API docs (Swagger/OpenAPI).
  - Benchmarks: Table of inference time (e.g., `| Batch Size | Latency | AUC |`), drift detection example.
  - Assumptions (e.g., English-only; KS threshold 0.05).
- **Trained model file** (ONNX/pickle) OR link to HuggingFace Hub repository (if hosted externally).
- Sample API responses.
- **Tests:** Pytest/Jest for API/ML (cover 80%).

Evaluation Criteria (~150 points – Scaled for ML Depth)

Category	Points	Key Focus Areas
ML Model & Pipeline	50	Fine-tuning quality, imbalance handling, AUC/F1 >0.92
Inference & Scale	30	<200ms latency, batching, ONNX opts
Backend Integration	30	API robustness, Mongo indexing, async queue
Drift/Explainability	20	KS monitoring, SHAP/LIME outputs
Code Quality/Docs	20	Modular (services), benchmarks explained

Bonus (Optional +20):

- Multi-language support (mBERT fine-tune).
- Human-in-loop: API for moderator overrides (update model via active learning).
- Deploy: Docker + FastAPI uvicorn; latency graphs.

Embed Square Solutions Pvt. Ltd.

Gokhale Business Bay, A-1111, Paschimnagari, Kothrud, Pune - 411038, Maharashtra, India.
www.embeds2.com

Important Notes

- Use free Hugging Face Hub for model push (optional).

- Document ethics: Bias audit on demographics.
- Focus on working end-to-end: Train → Infer → Alert.
- If local GPU unavailable, use Colab for training (upload dataset).

Good luck—build the shield against fake reviews plaguing local marketplaces!

Embed Square Solutions Pvt. Ltd.

Gokhale Business Bay, A-1111, Paschimnagari, Kothrud, Pune - 411038, Maharashtra, India.
www.embeds2.com